



(CSE 4802: Compiler Design Lab)

LAB 3

Report on YACC Parser for Scientific Calculator Expressions

Submitted by:

Md. Mosharrifur Rahman Ratul

Student ID: 210041213

Section: 2A

BSc in CSE, Dept of Computer Science and Engineering
Islamic University of Technology (IUT)

June 6, 2026

Contents

1	Introduction	2
2	Approach	2
3	Code Implementation	2
3.1	Lexical Analyzer (<code>lexer.l</code>)	2
3.2	Parser (<code>parser.y</code>)	4
4	Input and Output Files	9
4.1	Input File	9
4.2	Output File	10
5	Conclusion	10

1 Introduction

A parser is a component of a compiler that checks whether a sequence of tokens follows the grammar rules of a language. It receives tokens from the lexical analyzer and builds the syntactic structure of the input.

Flex is a tool used to generate lexical analyzers, while Yacc (Yet Another Compiler Compiler) is used to generate parsers from a given grammar. In this lab, Flex and Yacc are used together to implement a scientific calculator capable of evaluating mathematical expressions and functions.

2 Approach

The lexical analyzer is implemented using Flex to recognize numbers, operators, functions, constants, and identifiers. The parser is implemented using Yacc to define the grammar and evaluation rules of mathematical expressions.

The parser supports arithmetic operations, scientific functions, constants, variable assignments, and error handling. Input expressions are read from a file and the calculated results are written to an output file. I also checked the robustness of the YACC parser by intentionally giving wrong input and the parser can successfully detect them and report the errors properly.

3 Code Implementation

3.1 Lexical Analyzer (**lexer.l**)

Here is the code for lexer.l file.

Listing 1: lexer.l

```
1  %{
2
3
4  #include "parser.tab.h"
5  #include <stdlib.h>
6  #include <string.h>
7
8  %}
```

```

9
10 %%
11
12 [0-9]+(\.[0-9]+)? {
13     yylval.dval = atof(yytext);
14     return NUMBER;
15 }
16
17 PI      { return PI; }
18 E       { return E; }
19
20 sin     { return SIN; }
21 cos     { return COS; }
22 tan     { return TAN; }
23 sqrt    { return SQRT; }
24 log     { return LOG; }
25
26 deg     { return DEG; }
27 rad     { return RAD; }
28
29 [A-Za-z_][A-Za-z0-9_]* {
30     strcpy(yylval.sval, yytext);
31     return ID;
32 }
33
34 "+"     { return '+'; }
35 "-"     { return '-'; }
36 "*"     { return '*'; }
37 "/"     { return '/'; }
38 "^"     { return '^'; }
39 "="     { return '='; }
40
41 "("     { return '('; }
42 ")"     { return ')'; }
43
44 \r?\n   { return '\n'; }
45
46 [ \t]+  ;
47
48 .       { return yytext[0]; }
49
50 %%
51
52 int yywrap()
53 {
54     return 1;
55 }

```

3.2 Parser (`parser.y`)

Listing 2: `parser.y`

```
1
2 %{
3
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <math.h>
7 #include <string.h>
8
9 void yyerror(char *s);
10 int yylex();
11
12 extern FILE *yyin;
13 extern FILE *yyout;
14
15 #define MAX_VAR 100
16
17 typedef struct
18 {
19     char name[50];
20     double value;
21 } Variable;
22
23 Variable vars[MAX_VAR];
24 int var_count = 0;
25
26 double getVar(char *name)
27 {
28     for(int i=0; i<var_count; i++)
29         if(strcmp(vars[i].name, name)==0)
30             return vars[i].value;
31
32     fprintf(yyout, "Undefined variable: %s\n", name);
33     return 0;
34 }
35
36 void setVar(char *name, double val)
37 {
38     for(int i=0; i<var_count; i++)
39     {
40         if(strcmp(vars[i].name, name)==0)
41         {
42             vars[i].value=val;
43             return;
44         }
45     }
46 }
```

```

47     strcpy(vars[var_count].name,name);
48     vars[var_count].value=val;
49     var_count++;
50 }
51
52 int error_flag=0;
53 int semantic_error=0;
54 int fatal_error=0;
55
56 %}
57
58 %union
59 {
60     double dval;
61     char sval[50];
62 }
63
64 %token <dval> NUMBER
65 %token <sval> ID
66
67 %token SIN COS TAN SQRT LOG
68 %token PI E
69 %token DEG RAD
70
71 %type <dval> expr term power factor
72
73 %left '+' '-'
74 %left '*' '/'
75 %right '^'
76 %right UMINUS
77
78 %%
79
80 program:
81     lines
82 ;
83
84 lines:
85     lines line
86 |
87 ;
88
89 line:
90
91     ID '=' expr '\n'
92     {
93         setVar($1,$3);
94
95         if(!fatal_error)
96             fprintf(yyout,"%s = %lf\n", $1,$3);

```

```

97         error_flag=0;
98         semantic_error=0;
99         fatal_error=0;
100     }
101
102
103 | expr '\n'
104 {
105     if(!error_flag && !semantic_error && !fatal_error)
106         fprintf(yyout, "Result = %lf\n", $1);
107
108     error_flag=0;
109     semantic_error=0;
110     fatal_error=0;
111 }
112
113 | error '\n'
114 {
115     yyerrok;
116
117     error_flag=0;
118     semantic_error=0;
119     fatal_error=0;
120 }
121
122 | '\n'
123 ;
124
125 expr:
126
127     expr '+' term
128     { $$=$1+$3; }
129
130 | expr '-' term
131     { $$=$1-$3; }
132
133 | term
134     { $$=$1; }
135 ;
136
137 term:
138
139     term '*' power
140     { $$=$1*$3; }
141
142 | term '/' power
143     {
144         if($3==0)
145         {
146             fprintf(yyout, "Division by zero\n");

```

```

147         fatal_error=1;
148         $$=0;
149     }
150     else
151         $$=$1/$3;
152 }
153
154 | power
155 { $$=$1; }
156 ;
157
158 power:
159
160     factor '^' power
161     { $$=pow($1,$3); }
162
163 | factor
164 { $$=$1; }
165 ;
166
167 factor:
168
169     '-' factor %prec UMINUS
170     { $$=-$2; }
171
172 | SIN '(' expr ')'
173     { $$=sin($3); }
174
175 | COS '(' expr ')'
176     { $$=cos($3); }
177
178 | TAN '(' expr ')'
179     {
180         double c=cos($3);
181
182         if(fabs(c)<1e-10)
183         {
184             fprintf(yyout,"Undefined tan()\n");
185             fatal_error=1;
186             $$=0;
187         }
188         else
189             $$=tan($3);
190     }
191
192 | SQRT '(' expr ')'
193     {
194         if($3<0)
195         {
196             fprintf(yyout,"Invalid sqrt() domain\n");

```



```

197         fatal_error=1;
198         $$=0;
199     }
200     else
201         $$=sqrt($3);
202 }
203
204 | LOG '(' expr ')'
205 {
206     if($3<=0)
207     {
208         fprintf(yyout,"Invalid log() domain\n");
209         fatal_error=1;
210         $$=0;
211     }
212     else
213         $$=log($3);
214 }
215
216 | DEG '(' expr ')'
217 {
218     $$ = $3 * 180.0 / M_PI;
219 }
220
221 | RAD '(' expr ')'
222 {
223     $$ = $3 * M_PI / 180.0;
224 }
225
226 | '(' expr ')'
227 { $$=$2; }
228
229 | PI
230 { $$=M_PI; }
231
232 | E
233 { $$=M_E; }
234
235 | ID
236 {
237     $$=getVar($1);
238 }
239
240 | NUMBER
241 { $$=$1; }
242 ;
243
244 %%
245
246 void yyerror(char *s)

```

```

247 {
248     if(!fatal_error)
249         fprintf(yyout, "Syntax Error\n");
250
251     error_flag=1;
252 }
253
254 int main()
255 {
256     yyin=fopen("input.txt", "r");
257
258     if(!yyin)
259     {
260         printf("Cannot open input.txt\n");
261         return 1;
262     }
263
264     yyout=fopen("output.txt", "w");
265
266     if(!yyout)
267     {
268         printf("Cannot create output.txt\n");
269         return 1;
270     }
271
272     yyparse();
273
274     fclose(yyin);
275     fclose(yyout);
276
277     printf("Calculation completed. Check output.txt\n");
278
279     return 0;
280 }

```

4 Input and Output Files

4.1 Input File

The following input file was used to test the scientific calculator parser.

```

1 2+3*4
2 PI
3 E
4 -5+3
5 sin(PI/2)

```

```

6 sqrt(25
7 sqrt(-9)
8 log(10)
9 log(0)
10 x = 10
11 y = x + 5
12 y * 2
13 rad(180)
14 deg(3.14159265)
15 tan(PI/2)

```

4.2 Output File

The following output was generated by the parser.

```

1 Result = 14.000000
2 Result = 3.141593
3 Result = 2.718282
4 Result = -2.000000
5 Result = 1.000000
6 Syntax Error
7 Invalid sqrt() domain
8 Result = 2.302585
9 Invalid log() domain
10 x = 10.000000
11 y = 15.000000
12 Result = 30.000000
13 Result = 3.141593
14 Result = 180.000000
15 Undefined tan()

```

5 Conclusion

Here, the parser can successfully parse the scientific expressions. Here, it can also report various kinds of error. Such as, from the given input and output files we can see the parser can detect the division by zero, syntax error, etc. It can also check the wrong domain, like square root of negative number, $\log(0)$, $\tan(90)$ etc values and report error. So, this parser can successfully parse any scientific expressions.